Certainly! Below is a comprehensive guide to the `math` library in Python, including the constants and functions, with example usage for each.

Constants

math.pi

The mathematical constant π.

```
python
Copy code
import math

print(math.pi)  # Output: 3.141592653589793
```

math.e

The mathematical constant e.

```
python
Copy code
print(math.e)  # Output: 2.718281828459045
```

math.tau

The mathematical constant τ (2π).

```
python
Copy code
print(math.tau)  # Output: 6.283185307179586
```

math.inf

Floating-point positive infinity.

```
python
Copy code
print(math.inf)  # Output: inf
```

math.nan

Not a number (NaN).

```
python
Copy code
print(math.nan)  # Output: nan
```

Functions

Number-theoretic and representation functions

math.ceil(x)

Return the ceiling of x, the smallest integer greater than or equal to x.

```
python
Copy code
print(math.ceil(4.2))  # Output: 5
```

math.copysign(x, y)

Return a float with the magnitude (absolute value) of x but the sign of y.

```
python
Copy code
print(math.copysign(-3, 2))  # Output: 3.0
```

math.fabs(x)

Return the absolute value of x.

```
python
Copy code
print(math.fabs(-4.5))  # Output: 4.5
```

math.factorial(x)

Return x factorial.

```
python
Copy code
print(math.factorial(5))  # Output: 120
```

math.floor(x)

Return the floor of x, the largest integer less than or equal to x.

```
python
Copy code
print(math.floor(4.9))  # Output: 4
```

math.fmod(x, y)

Return the remainder of x / y.

```
python
Copy code
print(math.fmod(7, 3))  # Output: 1.0
```

math.frexp(x)

Return the mantissa and exponent of x as the pair (m, e).

```
python
Copy code
print(math.frexp(8))  # Output: (0.5, 4)
```

math.fsum(iterable)

Return an accurate floating point sum of values in the iterable.

```
python
Copy code
print(math.fsum([0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1,
0.1]))  # Output: 1.0
```

math.gcd(x, y)

Return the greatest common divisor of the integers x and y.

```
python
Copy code
print(math.gcd(48, 180))  # Output: 12
```

*math.isclose(a, b, *, rel_tol=1e-09, abs_tol=0.0)*

Determine whether two floating point numbers are close in value.

```
python
Copy code
print(math.isclose(1.0000001, 1.0000002))  # Output: True
```

math.isfinite(x)

Return True if x is neither an infinity nor a NaN.

```
python
Copy code
print(math.isfinite(2.5))  # Output: True
```

```
print(math.isfinite(math.inf))  # Output: False
```

math.isinf(x)

Return True if x is a positive or negative infinity.

```
python
Copy code
print(math.isinf(math.inf))  # Output: True
```

math.isnan(x)

Return True if x is a NaN (not a number).

```
python
Copy code
print(math.isnan(math.nan))  # Output: True
```

math.isqrt(n)

Return the integer square root of the nonnegative integer n.

```
python
Copy code
print(math.isqrt(10))  # Output: 3
```

math.ldexp(x, i)

Return x * (2**i).

```
python
Copy code
print(math.ldexp(5, 3))  # Output: 40.0
```

math.modf(x)

Return the fractional and integer parts of x.

```
python
Copy code
print(math.modf(4.25))  # Output: (0.25, 4.0)
```

*math.prod(iterable, *, start=1)*

Calculate the product of all the elements in the input iterable. The default start value for the product is 1.

```
python
Copy code
```

```
print(math.prod([1, 2, 3, 4]))  # Output: 24
```

math.trunc(x)

Return the Real value x truncated to an Integral (usually an integer).

```
python
Copy code
print(math.trunc(4.5))  # Output: 4
```

Power and logarithmic functions

math.exp(x)

Return e raised to the power of x.

```
python
Copy code
print(math.exp(2))  # Output: 7.38905609893065
```

math.expm1(x)

Return e raised to the power of x, minus 1.

```
python
Copy code
print(math.expm1(1))  # Output: 1.718281828459045
```

Tkinter (GUI Programming)

Tkinter is a graphical user interface (GUI) module for Python, you can make desktop apps with Python. You can make windows, buttons, show text and images amongst other things.

Tk and Tkinter apps can run on most Unix platforms. This also works on Windows and Mac OS X.
The module Tkinter is an interface to the Tk GUI toolkit.

Related course:

Example

Tkinter module

This example opens a blank desktop window. The tkinter module is part of the standard library. To use tkinter, import the tkinter module.

```
from tkinter import *
```

This is tkinter with underscore t, it has been renamed in Python 3.

Setup the window

Start tk and create a window.

```
root = Tk()
app = Window(root)
```

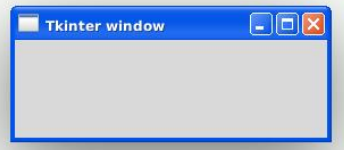
The window class is not standard, we create a Window. This class in itself is pretty basic.

```
class Window(Frame):
    def __init__(self, master=None):
        Frame.__init__(self, master)
        self.master = master
```

Then set the window title and show the window:

```
# set window title
root.wm_title("Tkinter window")
```

```
# show window
root.mainloop()
```



Tkinter window example

The program below shows an empty tkinter window.
Run with the program below:

```
from tkinter import *

class Window(Frame):
    def __init__(self, master=None):
        Frame.__init__(self, master)
        self.master = master

# initialize tkinter
root = Tk()
app = Window(root)

# set window title
root.wm_title("Tkinter window")

# show window
root.mainloop()
```

```
from tkinter import *
root = Tk()
w = Label(root, text='HELLO1234567.org!')
w.pack()
root.mainloop()
```

```
from tkinter import *

master = Tk()
Label(master, text='First Name').grid(row=0)
Label(master, text='Last Name').grid(row=1)
e1 = Entry(master)
e2 = Entry(master)
e1.grid(row=0, column=1)
e2.grid(row=1, column=1)
mainloop()
```

from tkinter import *

master = Tk()

var1 = IntVar()

Checkbutton(master, text='male', variable=var1).grid(row=0, sticky=W)

var2 = IntVar()

Checkbutton(master, text='female', variable=var2).grid(row=1, sticky=W)

Mainloop()

```
from tkinter import *

root = Tk()
menu = Menu(root)
root.config(menu=menu)
filemenu = Menu(menu)
menu.add_cascade(label='File', menu=filemenu)
filemenu.add_command(label='New')
filemenu.add_command(label='Open...')
filemenu.add_separator()
filemenu.add_command(label='Exit', command=root.quit)
helpmenu = Menu(menu)
menu.add_cascade(label='Help', menu=helpmenu)
helpmenu.add_command(label='About')
mainloop()
```

------PANDAS

Python Pandas Tutorial

The term "Pandas" refers to an open-source library for manipulating high-performance data in Python. This instructional exercise is intended for the two novices and experts.

It was created in 2008 by Wes McKinney and is used for data analysis in Python. Pandas is an open-source library that provides high-performance data manipulation in Python. All of the basic and advanced concepts of Pandas, such as Numpy, data operation, and time series, are covered in our tutorial.

Pandas Introduction

The name of Pandas is gotten from the word Board Information, and that implies an Econometrics from Multi-faceted information. It was created in 2008 by Wes McKinney and is used for data analysis in Python.

- It has a DataFrame object that is quick and effective, with both standard and custom indexing.
- Utilized for reshaping and turning of the informational indexes.
- For aggregations and transformations, group by data.

1) Series

A one-dimensional array capable of storing a variety of data types is how it is defined. The term "index" refers to the row labels of a series. We can without much of a stretch believer the rundown, tuple, and word reference into series utilizing "series' technique. Multiple columns cannot be included in a Series. Only one parameter exists:

Data: It can be any list, dictionary, or scalar value.

Creating Series from Array:

Before creating a Series, Firstly, we have to import the numpy module and then use array() function in the program.

```
import pandas as pd
import numpy as np
info = np.array(['P','a','n','d','a','s'])
a = pd.Series(info)
print(a)
```

Output

```
0   P
1   a
2   n
3   d
4   a
5   s
dtype: object
```

Explanation: In this code, firstly, we have imported the **pandas** and **numpy** library with the **pd** and **np** alias. Then, we have taken a variable named "info" that consist of an array of some values. We have called the **info** variable through a **Series** method and defined it in an "**a**" variable. The Series has printed by calling the **print(a)** method.

Python Pandas DataFrame

It is a generally utilized information design of pandas and works with a two-layered exhibit with named tomahawks (lines and segments). As a standard method for storing data, DataFrame has two distinct indexes-row index and column index. It has the following characteristics:

- The sections can be **heterogeneous** sorts like int, bool, etc.

It can be thought of as a series structure dictionary with indexed rows and columns. It is referred to as "columns" for rows and "index" for columns.

Create a DataFrame using List:

We can easily create a DataFrame in Pandas using list.

```
import pandas as pd
# a list of strings
```

```
x = ['Python', 'Pandas']

# Calling DataFrame constructor on list
df = pd.DataFrame(x)
print(df)
```

Output

```
      0
0   Python
1   Pandas
```

Explanation: In this code, we have characterized a variable named "x" that comprise of string values. On a list, the values are being printed by calling the DataFrame constructor.

Python Pandas Series

The Pandas Series can be defined as a one-dimensional array that is capable of storing various data types. We can easily convert the list, tuple, and dictionary into series using "**series**' method. The row labels of series are called the index. A Series cannot contain multiple columns. It has the following parameter:

- **data:** It can be any list, dictionary, or scalar value.
- **index:** The value of the index should be unique and hashable. It must be of the same length as data. If we do not pass any index, default **np.arrange(n)** will be used.
- **dtype:** It refers to the data type of series.
- **copy:** It is used for copying the data.

Creating a Series:

We can create a Series in two ways:

1. Create an empty Series
2. Create a Series using inputs.

Create an Empty Series:

We can easily create an empty series in Pandas which means it will not have any value.

The syntax that is used for creating an Empty Series:

```
<series object> = pandas.Series()
```

The below example creates an Empty Series type object that has no values and having default datatype, i.e., **float64**.

Example

```
import pandas as pd
x = pd.Series()
print (x)
```

Output

```
Series([], dtype: float64)
```

Creating a Series using inputs:

We can create Series by using various inputs:

- Array
- Dict
- Scalar value

Creating Series from Array:

Before creating a Series, firstly, we have to import the **numpy** module and then use array() function in the program. If the data is ndarray, then the passed index must be of the same length.

If we do not pass an index, then by default index of **range(n)** is being passed where n defines the length of an array, i.e., [0,1,2,....**range(len(array))-1**].

Example

```
import pandas as pd
import numpy as np
```

3. info = np.array(['P','a','n','d','a','s'])
4. a = pd.Series(info)
5. print(a)

Output

```
0    P
1    a
2    n
3    d
4    a
5    s
dtype: object
```

Create a Series from dict

We can also create a Series from dict. **If the dictionary object is being passed as an input and the index is not specified, then the dictionary keys are taken in a sorted order to construct the index**.

If index is passed, then values correspond to a particular label in the index will be extracted from the **dictionary**.

1. #**import** the pandas library
2. **import** pandas as pd
3. **import** numpy as np
4. info = {'x' : 0., 'y' : 1., 'z' : 2.}
5. a = pd.Series(info)
6. print (a)

Output

```
x    0.0
y    1.0
z    2.0
dtype: float64
```

Create a Series using Scalar:

If we take the scalar values, then the index must be provided. The scalar value will be repeated for matching the length of the index.

1. #**import** pandas library

```
import pandas as pd
import numpy as np
x = pd.Series(4, index=[0, 1, 2, 3])
print (x)
```

Output

```
0       4
1       4
2       4
3       4
dtype: int64
```

Accessing data from series with Position:

Once you create the Series type object, you can access its indexes, data, and even individual elements.

The data in the Series can be accessed similar to that in the ndarray.

```
import pandas as pd
x = pd.Series([1,2,3],index = ['a','b','c'])
#retrieve the first element
print (x[0])
```

Output

```
1
```

Retrieving Index array and data array of a series object

We can retrieve the index array and data array of an existing Series object by using the attributes index and values.

```
import numpy as np
import pandas as pd
x=pd.Series(data=[2,4,6,8])
y=pd.Series(data=[11.2,18.6,22.5], index=['a','b','c'])
print(x.index)
print(x.values)
```

7. print(y.index)
8. print(y.values)

Output

```
RangeIndex(start=0, stop=4, step=1)
[2 4 6 8]
Index(['a', 'b', 'c'], dtype='object')
[11.2 18.6 22.5]
```

Retrieving Types (dtype) and Size of Type (itemsize)

You can use attribute dtype with Series object as <objectname> dtype for retrieving the data type of an individual element of a series object, you can use the **itemsize** attribute to show the number of bytes allocated to each data item.

```
1. import numpy as np
2. import pandas as pd
3. a=pd.Series(data=[1,2,3,4])
4. b=pd.Series(data=[4.9,8.2,5.6],
5. index=['x','y','z'])
6. print(a.dtype)
7. print(a.itemsize)
8. print(b.dtype)
9. print(b.itemsize)
```

Output

```
int64
8
float64
8
```

Retrieving Shape

The shape of the Series object defines total number of elements including missing or empty values(NaN).

```
1. import numpy as np
2. import pandas as pd
3. a=pd.Series(data=[1,2,3,4])
```

4. b=pd.Series(data=[4.9,8.2,5.6],index=['x','y','z'])
5. print(a.shape)
6. print(b.shape)

Output

```
(4,)
(3,)
```

Pandas Series.to_frame()

Series is defined as a type of list that can hold an integer, string, double values, etc. It returns an object in the form of a list that has an index starting from 0 to n where n represents the length of values in Series.

The main difference between Series and Data Frame is that **Series can only contain a single list with a particular index, whereas the DataFrame is a combination of more than one series that can analyze the data**.

The Pandas **Series.to_frame()** function is used to convert the series object to the DataFrame.

Syntax

1. Series.to_frame(name=None)

Parameters

name: Refers to the object. Its Default value is None. If it has one value, the passed name will be substituted for the series name.

Returns

It returns DataFrame representation of Series.

Example1

1. s = pd.Series(["a", "b", "c"],

2. name="vals")
3. s.to_frame()

Output

```
       vals
0          a
1          b
2          c
```

Example2

```
1. import pandas as pd
2. import matplotlib.pyplot as plt
3. emp = ['Parker', 'John', 'Smith', 'William']
4. id = [102, 107, 109, 114]
5. emp_series = pd.Series(emp)
6. id_series = pd.Series(id)
7. frame = { 'Emp': emp_series, 'ID': id_series }
8. result = pd.DataFrame(frame)
9. print(result)
```

Output

```
      Emp        ID
0   Parker      102
1   John         107
2   Smith       109
3   William    114
```

PROBLEMS

```
import pandas as pd

mydataset = {

 'cars': ["BMW", "Volvo", "Ford"],

 'passings': [3, 7, 2]

}

myvar = pd.DataFrame(mydataset)
```

print(myvar) o/p->

	cars	passings
0	BMW	3
1	Volvo	7
2	Ford	2

import pandas as pd

df = pd.read_csv('data.csv')

print(df.to_string()))

```
print(df.to_string())                                                                                    Duration  Pulse  Maxpulse
Calories
0           60    110        130      409.1
1           60    117        145      479.0
2           60    103        135      340.0
3           45    109        175      282.4
4           45    117        148      406.0
5           60    102        127      300.5
6           60    110        136      374.0
7           45    104        134      253.3
8           30    109        133      195.1
9           60     98        124      269.0
10          60    103        147      329.3
11          60    100        120      250.7
12          60    106        128      345.3
13          60    104        132      379.3
14          60     98        123      275.0
```

```
import pandas as pd

data = {
  "Duration":{
    "0":60,
    "1":60,
    "2":60,
    "3":45,
    "4":45,
    "5":60
  },
  "Pulse":{
    "0":110,
    "1":117,
    "2":103,
    "3":109,
    "4":117,
    "5":102
  },
  "Maxpulse":{
    "0":130,
    "1":145,
    "2":135,
    "3":175,
    "4":148,
    "5":127
  },
  "Calories":{
    "0":409.1,
    "1":479.0,
    "2":340.0,
    "3":282.4,
    "4":406.0,
    "5":300.5
  }
}

df = pd.DataFrame(data)

print(df)
```

```
   Duration  Pulse  Maxpulse  Calories
0        60    110       130     409.1
1        60    117       145     479.0
2        60    103       135     340.0
3        45    109       175     282.4
4        45    117       148     406.0
5        60    102       127     300.5
```

import pandas as pd

df = pd.read_csv('data.csv')

print(df.head())

```
   Duration  Pulse  Maxpulse  Calories
0        60    110       130     409.1
1        60    117       145     479.0
2        60    103       135     340.0
3        45    109       175     282.4
4        45    117       148     406.0
```

Matplotlib (Python Plotting Library)

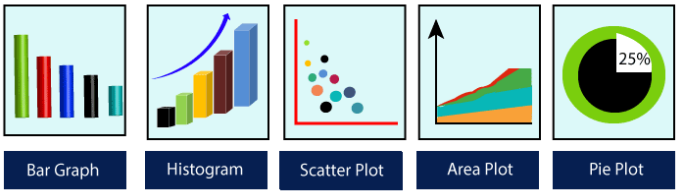
Human minds are more adaptive for the visual representation of data rather than textual data. We can easily understand things when they are visualized. It is better to represent the data through the graph where we can analyze the data more efficiently and make the specific decision according to data analysis. Before learning the matplotlib, we need to understand data visualization and why data visualization is important.

Data Visualization

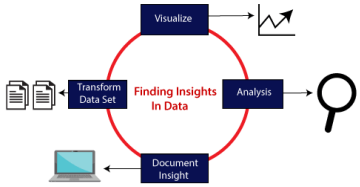
Graphics provides an excellent approach for exploring the data, which is essential for presenting results. Data visualization is a new term. It expresses the idea that involves more than just representing data in the graphical form (instead of using textual form).

This can be very helpful when discovering and getting to know a dataset and can help with classifying patterns, corrupt data, outliers, and much more. With a little domain knowledge, data visualizations can be used to express and demonstrate key relationships in plots and charts. The static does indeed focus on quantitative description and estimations of data. It provides an important set of tools for gaining a qualitative understanding.

There are five key plots that are used for data visualization.



There are five phases which are essential to make the decision for the organization:



- **Visualize:** We analyze the raw data, which means it makes complex data more accessible, understandable, and more usable. Tabular data representation is used where the user will look up a specific measurement, while the chart of several types is used to show patterns or relationships in the data for one or more variables.
- **Analysis:** Data analysis is defined as cleaning, inspecting, transforming, and modeling data to derive useful information. Whenever we make a decision for the business or in daily life, is by past experience. **What will happen to choose a particular decision**, it is nothing but analyzing our past. That may be affected in the future, so the proper analysis is necessary for better decisions for any business or organization.
- **Document Insight:** Document insight is the process where the useful data or information is organized in the document in the standard format.
- **Transform Data Set:** Standard data is used to make the decision more effectively.

Why need data visualization?

Data visualization can perform below tasks:

- It identifies areas that need improvement and attention.
- It clarifies the factors.
- It helps to understand which product to place where.
- Predict sales volumes.

Benefit of Data Visualization

Here are some benefits of the data visualization, which helps to make an effective decision for the organizations or business:

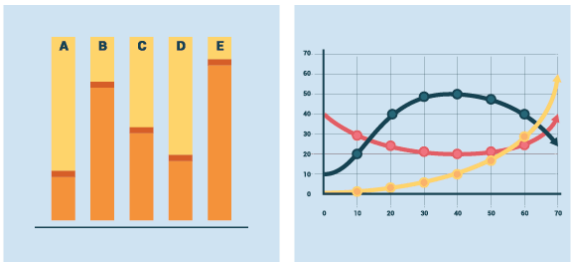
1. Building ways of absorbing information

Data visualization allows users to receive vast amounts of information regarding operational and business conditions. It helps decision-makers to see the relationship between multi-dimensional data sets. It offers new ways to analyses data through the use of maps, fever charts, and other rich graphical representations.

Visual data discovery is more likely to find the information that the organization needs and then end up with being more productive than other competitive companies.

2. Visualize relationship and patterns in Businesses

The crucial advantage of data visualization is that it is essential to find the correlation between operating conditions and business performance in today's highly competitive business environment.



The ability to make these types of correlations enables the executives to identify the root cause of the problem and act quickly to resolve it.

Suppose a food company is looking their monthly customer data, and the data is presented with bar charts, which shows that the company's score has dropped by five points in the previous months in that particular region; the data suggest that there's a problem with customer satisfaction in this area.

3. Take action on the emerging trends faster

Data visualization allows the decision-maker to grasp shifts in customer behavior and market conditions across multiple data sets more efficiently.

Having an idea about the customer's sentiments and other data discloses an emerging opportunity for the company to act on new business opportunities ahead of their competitor.

4. Geological based Visualization

Geo-spatial visualization is occurred due to many websites providing web-services, attracting visitor's interest. These types of websites are required to take benefit of location-specific information, which is already present in the customer details.

Matplotlib is a Python library which is defined as a multi-platform data visualization library built on Numpy array. It can be used in python scripts, shell, web application, and other graphical user interface toolkit.

The **John D. Hunter** originally conceived the matplotlib in **2002**. It has an active development community and is distributed under a **BSD-style license**. Its first version was released in 2003, and the latest **version 3.1.1** is released on **1 July 2019**.

Matplotlib 2.0.x supports Python versions 2.7 to 3.6 till 23 June 2007. Python3 support started with Matplotlib 1.2. Matplotlib 1.4 is the last version that supports Python 2.6.

There are various toolkits available that are used to enhance the functionality of the **matplotlib**. Some of these tools are downloaded separately, others can be shifted with the matplotlib source code but have external dependencies.

- **Bashmap:** It is a map plotting toolkit with several map projections, coastlines, and political boundaries.
- **Cartopy:** It is a mapping library consisting of object-oriented map projection definitions, and arbitrary point, line, polygon, and image transformation abilities.
- **Excel tools:** Matplotlib provides the facility to utilities for exchanging data with Microsoft Excel.
- **Mplot3d:** It is used for 3D plots.
- **Natgrid:** It is an interface to the Natgrid library for irregular gridding of the spaced data.

Matplotlib Architecture

There are three different layers in the architecture of the matplotlib which are the following:

- Backend Layer

- Artist layer
- Scripting layer

Backend layer

The backend layer is the bottom layer of the figure, which consists of the implementation of the various functions that are necessary for plotting. There are three essential classes from the backend layer **FigureCanvas**(The surface on which the figure will be drawn), **Renderer**(The class that takes care of the drawing on the surface), and **Event**(It handle the mouse and keyboard events).

Artist Layer

The artist layer is the second layer in the architecture. It is responsible for the various plotting functions, like axis, which coordinates on how to use the renderer on the figure canvas.

Scripting layer

The scripting layer is the topmost layer on which most of our code will run. The methods in the scripting layer, almost automatically take care of the other layers, and all we need to care about is the current state (figure & subplot).

The General Concept of Matplotlib

A Matplotlib figure can be categorized into various parts as below:

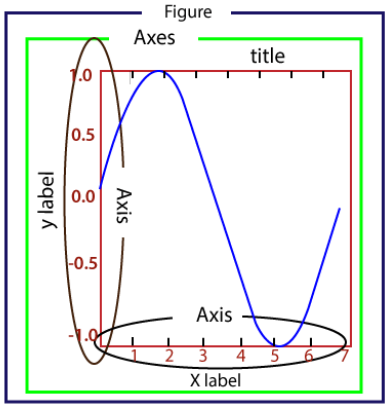


Figure: It is a whole figure which may hold one or more axes (plots). We can think of a Figure as a canvas that holds plots.

Axes: A Figure can contain several Axes. It consists of two or three (in the case of 3D) Axis objects. Each Axes is comprised of a title, an x-label, and a y-label.

Axis: Axises are the number of line like objects and responsible for generating the graph limits.

Artist: An artist is the all which we see on the graph like Text objects, Line2D objects, and collection objects. Most Artists are tied to Axes.

Installing Matplotlib

Before start working with the Matplotlib or its plotting functions first, it needs to be installed. The installation of matplotlib is dependent on the distribution that is installed on your computer. These installation methods are following:

Use the Anaconda distribution of Python

The easiest way to install Matplotlib is to download the Anaconda distribution of Python. Matplotlib is pre-installed in the anaconda distribution No further installation steps are necessary.

pip install matplotlib

Verify the Installation

To verify that matplotlib is installed properly or not, type the following command includes calling .__version __ in the terminal.

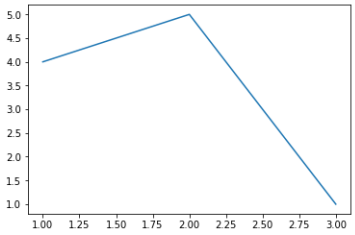
```
1. import matplotlib
2. matplotlib.__version__
3. '3.1.1'
```

Basic Example of plotting Graph

Here is the basic example of generating a simple graph; the program is following:

```
1. from matplotlib import pyplot as plt
2. #ploting our canvas
3. plt.plot([1,2,3],[4,5,1])
4. #display the graph
5. plt.show()
```

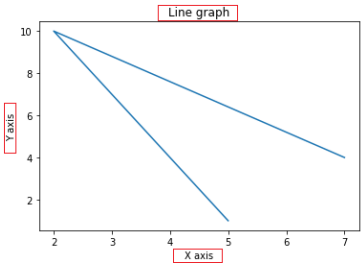
Output:



It takes only three lines to plot a simple graph using the Python matplotlib. We can add titles, labels to our chart which are created by Python matplotlib library to make it more meaningful. The example is the following:

```
1. from matplotlib import pyplot as plt
2.
3. x = [5, 2, 7]
4. y = [1, 10, 4]
5. plt.plot(x, y)
6. plt.title('Line graph')
7. plt.ylabel('Y axis')
8. plt.xlabel('X axis')
9. plt.show()
```

Output:



The graph is more understandable from the previous graph

Working with Pyplot

The **matplotlib.pyplot** is the collection command style functions that make matplotlib feel like working with MATLAB. The pyplot functions are used to make some changes to figure such as create a figure, creates a plotting area in a figure, plots some lines in a plotting area, decorates the plot including labels, etc.

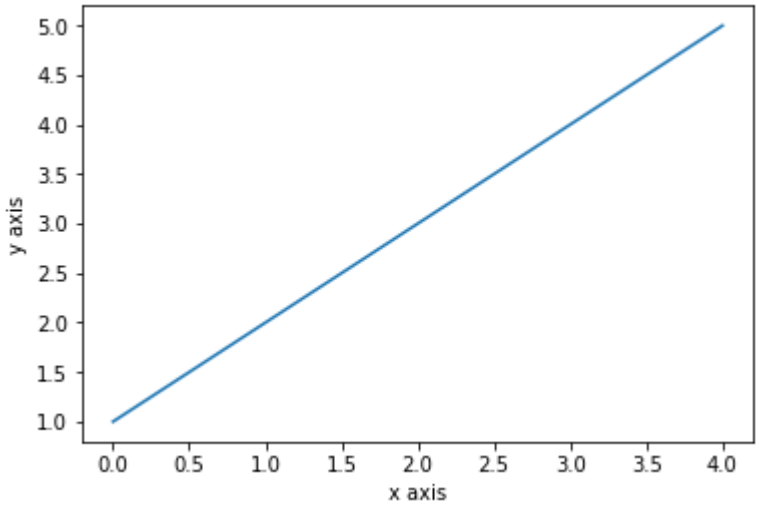
It is good to use when we want to plot something quickly without instantiating any figure or Axes.

While working with **matplotlib.pyplot**, some states are stored across function calls so that it keeps track of the things like current figure and plotting area, and these plotting functions are directed to the current axes.

The pyplot module provide the **plot()** function which is frequently use to plot a graph. Let's have a look on the simple example:

```
1. from matplotlib import pyplot as plt
2. plt.plot([1,2,3,4,5])
3. plt.ylabel("y axis")
4. plt.xlabel('x axis')
5. plt.show()
```

Output:

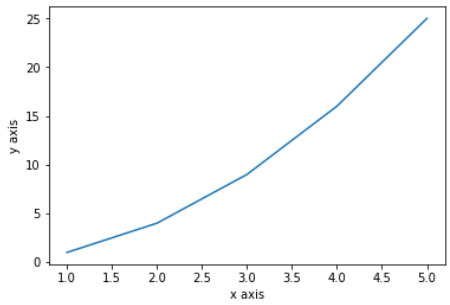


In the above program, it plots the graph x-axis ranges from 0-4 and the y-axis from 1-5. If we provide a single list to the plot(), matplotlib assumes it is a sequence of y values, and automatically generates the x values. Since we know that python index starts at 0, the default x vector has the same length as y but starts at 0. Hence the x data are [0, 1, 2, 3, 4].

We can pass the arbitrary number of arguments to the plot(). For example, to plot x versus y, we can do this following way:

```
1. from matplotlib import pyplot as plt
2. plt.plot([1,2,3,4,5],[1,4,9,16,25])
3. plt.ylabel("y axis")
4. plt.xlabel('x axis')
5. plt.show()
```

Output:



Formatting the style of the plot

There is an optional third argument, which is a format string that indicates the color and line type of the plot. The default format string is '**b-**'which is the solid blue as you can observe in the above plotted graph. Let's consider the following example where we plot the graph with the red circle.

```
1. from matplotlib import pyplot as plt
2. plt.plot([1, 2, 3, 4,5], [1, 4, 9, 16,25], 'ro')
3. plt.axis([0, 6, 0, 20])
4. plt.show()
```

Output: